

Assignment A1 - Anthony Mortimer - CSC 410 DT1

Times

Algorithm	Size	Time: Real	Time: User	Time: System
Array Sum	1,000,000,000	0m7.016s	0m4.001s	0m3.015s
Matrix Mul.	1,200	0m15.674s	0m15.664s	0m0.010s
N Queens	15	0m47.608s	0m47.600s	0m0.005s
Sorting	60,000	0m11.754s	0m11.582s	0m0.033s
Trapezoidal Rule	2,140,000,000	0m5.495s	0m5.493s	0m0.001s

Array Sum

```
arraysum.c x README.txt M x
sequential > arraysum.c > ...
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define SIZE 1000000000
5
6  long long sumArray(int arr[], int size)
7  {
8      long long sum = 0;
9
10     for (int i = 0; i < size; i++)
11     {
12         sum += arr[i];
13     }
14     return sum;
15 }
16
17 int main()
18 {
19     // int arr[SIZE];
20     int *arr = (int*)malloc(SIZE * sizeof(int));
21
22     for (int i = 0; i < SIZE; i++)
23     {
24         arr[i] = i + 1;
25     }
26
27     long long totalSum = sumArray(arr, SIZE);
28     printf("Total Sum: %lld\n", totalSum);
29
30     return 0;
31 }

Problems Output Debug Console Terminal Ports 4
cd "/home/agn20/A1-Mortimer/sequential/" && gcc arraysum.c -o arraysum.out && "/home/agn20/A1-Mortimer/sequential/"arraysum.out
agn20@Desktop-5820:~/A1-Mortimer$ cd "/home/agn20/A1-Mortimer/sequential/" && gcc arraysum.c -o arraysum.out && "/home/agn20/A1-Mortimer/sequential/"arraysum.out
Total Sum: 500000000500000000
agn20@Desktop-5820:~/A1-Mortimer/sequential$
```

Matrix Multiplication

```
C mainM.c M X C matrixMul.h X
sequential > matrixMul > C mainM.c > main()
1 #include "matrixMul.h"
2
3 int main()
4 {
5     // Dynamically allocate memory for the matrices
6     int** A = (int**)malloc(N * sizeof(int*));
7     int** B = (int**)malloc(N * sizeof(int*));
8     int** C = (int**)malloc(N * sizeof(int*));
9
10    for (int i = 0; i < N; ++i) {
11        A[i] = (int*)malloc(N * sizeof(int));
12        B[i] = (int*)malloc(N * sizeof(int));
13        C[i] = (int*)malloc(N * sizeof(int));
14    }
15
16    if (A == NULL || B == NULL || C == NULL) {
17        printf("Memory allocation failed!\n");
18        return -1;
19    }
20
21    printf("Matrices allocated successfully.\n");
22
23    // Initialize matrices A and B
24    for (int i = 0; i < N; ++i) {
25        for (int j = 0; j < N; ++j) {
26            A[i][j] = 1;
27            B[i][j] = 1;
28            C[i][j] = 0;
29        }
30    }
31}
```

N Queens

```
mainQ.c X
sequential > Nqueens > C mainQ.c > ...
1  #include "nqueens.h"
2
3  int main()
4  {
5      int n = 15;
6      int* board = (int*)malloc(n * sizeof(int));
7      if (board == NULL) {
8          fprintf(stderr, "Memory allocation failed\n");
9          return 1;
10     }
11
12     for (int i = 0; i < n; i++) {
13         board[i] = -1;
14     }
15
16     solveNQueensUtil(board, 0, n);
17     printBoard(board, n);
18
19     free(board);
20     return 0;
21 }
22
```

Problems Output Debug Console Terminal Ports 4

```
cd "/home/agm20/A1-Mortimer/sequential/Nqueens/" && gcc mainQ.c -o mainQ.out
● agm20@Desktop-5820:~/A1-Mortimer$ cd "/home/agm20/A1-Mortimer/sequential/Nqueens/"
| 14 12 10 13 11 6 4 2 0 9 7 9 6 11 7 |
○ agm20@Desktop-5820:~/A1-Mortimer/sequential/Nqueens$
```

Sorting

```
mainSorts.c X
sequential > sorts > mainSorts.c > ...
1  #include "sorts.h"
2  #include "sorts.c"
3
4  int main()
5  {
6      srand(time(NULL));
7
8      // Dynamically allocate arrays
9      int *arr1 = (int*)malloc(SIZE * sizeof(int));
10     int *arr2 = (int*)malloc(SIZE * sizeof(int));
11
12     if (!arr1 || !arr2) {
13         fprintf(stderr, "Memory allocation failed\n");
14         return 1;
15     }
16
17     // Populate arrays with random values
18     for (int i = 0; i < SIZE; i++) {
19         int val = rand() % MAX_VAL;
20         arr1[i] = val;
21         arr2[i] = val;
22     }
23
24     printf("Original array:\n");
25     printArray(arr1, SIZE);
26
27     mergeSort(arr1, 0, SIZE - 1);
28     printf("Sorted with Merge Sort:\n");
29     printArray(arr1, SIZE);
30
31     bubbleSort(arr2, SIZE);
```

Problems Output Debug Console Terminal Ports 4

```
9220 99222 99223 99229 99229 99231 99232 99239 99244 99244 99246 99247 99252 99254 99256 99257 99262 99264 99265 99266 99266 99273 99273 9
98 99300 99303 99304 99304 99307 99308 99309 99312 99315 99317 99318 99318 99320 99320 99321 99322 99322 99323 99324 99325 99327 99328 993
99361 99362 99365 99365 99366 99367 99369 99369 99375 99376 99377 99379 99380 99384 99384 99385 99395 99397 99398 99398 99399 99404 99405
9420 99420 99421 99422 99422 99428 99430 99431 99433 99434 99436 99437 99437 99439 99440 99440 99440 99444 99444 99445 99446 99447 99447 9
73 99473 99474 99474 99478 99479 99480 99480 99482 99483 99484 99485 99485 99486 99486 99488 99489 99490 99492 99494 99494 99495 99496 994
99516 99516 99517 99520 99522 99523 99526 99529 99531 99534 99540 99540 99541 99542 99544 99545 99547 99547 99548 99550 99550 99552 99555
9575 99578 99578 99581 99582 99584 99585 99586 99587 99589 99590 99592 99592 99594 99596 99601 99601 99602 99607 99612 99614 99614 99615 9
35 99636 99637 99639 99640 99642 99643 99644 99647 99648 99648 99650 99651 99653 99653 99653 99654 99659 99663 99665 99666 99671 996
99690 99691 99691 99693 99697 99697 99702 99702 99705 99707 99713 99714 99715 99716 99717 99720 99722 99723 99724 99724 99725 99727 99731
9744 99746 99746 99747 99751 99751 99752 99755 99759 99761 99762 99763 99764 99766 99766 99767 99771 99775 99777 99777 99780 99780 99781 9
94 99795 99797 99806 99807 99808 99809 99812 99814 99818 99820 99821 99822 99823 99825 99825 99825 99828 99830 99830 99830 99832 998
99862 99864 99865 99866 99867 99868 99872 99873 99876 99877 99878 99879 99879 99882 99884 99885 99885 99886 99887 99887 99887 99888 99888
9905 99909 99913 99915 99915 99916 99916 99917 99919 99919 99920 99920 99922 99923 99923 99924 99925 99926 99927 99929 99931 99931 99931 9
56 99958 99963 99964 99964 99965 99969 99970 99974 99975 99976 99976 99976 99978 99978 99981 99982 99983 99986 99987 99988 99989 99997
```

agm20@Desktop-5820:~/A1-Mortimer/sequential/sorts\$

Trapezoidal Rule

```
numIntegrate.c X
sequential > numIntegrate.c > ...
1  #include <stdio.h>
2
3  #define N 2140000000 // intervals
4
5  double f(double x) {
6      return 4.0 / (1.0 + x * x); // Function to integrate
7  }
8
9  double trapezoidalRule()
10 {
11     double a = 0;
12     double b = 1;
13     double h = (b - a) / N;
14     double sum = f(a) + f(b);
15
16     for (int i = 1; i < N; i++)
17     {
18         sum += 2 * f(a + i * h);
19     }
20
21     return sum * h / 2;
22 }
23
24 int main() {
25     double pi = trapezoidalRule();
26     printf("Estimated value of  $\pi$ : %f\n", pi);
27     return 0;
28 }
29
```

Problems Output Debug Console Terminal Ports 4

```
● agm20@Desktop-5820:~/A1-Mortimer/sequential/sorts$ cd "/home/agm20/A1-Mortimer/sequential/" &&
Estimated value of  $\pi$ : 3.141593
○ agm20@Desktop-5820:~/A1-Mortimer/sequential$
```